An Approach for Tiling and Fusing Dependent Reductions in Linalg

MLIR Developer Meetup - Oct 2026

Charitha Saumya
Intel MLIR Team, Intel Labs

intel.

Tile, Fuse Producers/Consumers, and Repeat

```
// prologue: A + c
%a = linalg.add ins(%A, %cst) outs(%e0)

// GEMM
%c = linalg.matmul ins(%a, %B) outs(%C)

// epilogue: exp
%d = linalg.exp ins(%c) outs(%D)
```

Tile the GEMM
Fuse producers
Fuse consumers



```
%r = scf.forall (%i, %j) = (0, 0) to (M, N)
       step (tM, tN) shared_outs(%o = %D) {
  %At = tensor.extract_slice %A[%i, 0] [tM, K]
  %at = linalg.add ins(%At, %cst) outs(%e0t)
  %Bt = tensor.extract_slice %B[0, %j] [K, tN]
  %Ct = tensor.extract_slice %C[%i, %j] [tM, tN]
  %ct = linalg.matmul ins(%at, %Bt) outs(%Ct)
  %Ot = tensor.extract_slice %o[%i, %j] [tM, tN]
  %dt = linalg.exp ins(%ct) outs(%Ot)
  scf.forall.in_parallel {
    tensor.parallel_insert_slice %dt into
        %o[%i, %j] [tM, tN]
  }
}
```

Tile, Fuse Producers/Consumers, and Repeat

```
// prologue: A + c
%a = linalg.add ins(%A, %cst) outs(%e0)

// GEMM
%c = linalg.matmul ins(%a, %B) outs(%C)

// epilogue: exp
%d = linalg.exp ins(%c) outs(%D)
```

Tile the GEMM
Fuse producers
Fuse consumers



```
%r = scf.forall (%i, %j) = (0, 0) to (M, N)
       step (tM, tN) shared_outs(%o = %D) {
  %At = tensor.extract_slice %A[%i, 0] [tM, K]
  %at = linalg.add ins(%At, %cst) outs(%e0t)
  %Bt = tensor.extract_slice %B[0, %j] [K, tN]
  %Ct = tensor.extract_slice %C[%i, %j] [tM, tN]
  %ct = linalg.matmul ins(%at, %Bt) outs(%Ct)
  %Ot = tensor.extract_slice %o[%i, %j] [tM, tN]
  %dt = linalg.exp ins(%ct) outs(%Ot)
  scf.forall.in_parallel {
    tensor.parallel_insert_slice %dt into
        %o[%i, %j] [tM, tN]
  }
}
```

Great support for tiling along parallel/reduction dims and fusion along parallel dims

How about Dependent Reductions?

```
// R1: reduce along the reduction dim
%m = linalg.reduce {red_kind1}
       ins(%x) outs(%init1)
       dimensions = [1]
// R1's result, broadcast along it
%p = linalg.generic {...}
       ins(%x, %m) outs(%init)
// R2: reduce along the same dim
%s = linalg.reduce {red_kind2}
       ins(%p) outs(%init2)
       dimensions = [1]
```

Tile R2
Fuse producers



```
%s = scf.for %j = 0 to N step tN iter_args(%a) {
  %xt = tensor.extract_slice %x[0, %j] [M, tN]
  // R1 fused at FULL extent: reads all of x
  %m  = linalg.reduce {red_kind1} ins(%x) outs(%i1)
  %pt = linalg.generic {...} ins(%xt, %m) outs(%et)
  %st = linalg.reduce {red_kind2} ins(%pt) outs(%a)
  scf.yield %st
}
```

R2's tile needs R1's result over the *whole* row ⇒ R1 can't be tiled
It re-reads all of `x` in *every* iteration ⇒ bad performance

Settle for Separate Loops

```
// R1: reduce along the reduction dim
%m = linalg.reduce {red_kind1}
       ins(%x) outs(%init1)
       dimensions = [1]
// R1's result, broadcast along it
%p = linalg.generic {...}
       ins(%x, %m) outs(%init)
// R2: reduce along the same dim
%s = linalg.reduce {red_kind2}
       ins(%p) outs(%init2)
       dimensions = [1]
```

Tile R1
Tile R2
No fusion



```
%m = scf.for %j = 0 to N step tN iter_args(%a) {
  %xt = tensor.extract_slice %x[0, %j] [M, tN]
  %mt = linalg.reduce {red_kind1} ins(%xt) outs(%a)
  scf.yield %mt
}
%s = scf.for %j = 0 to N step tN iter_args(%a) {
  %xt = tensor.extract_slice %x[0, %j] [M, tN]
  %pt = linalg.generic {...} ins(%xt, %m) outs(%et)
  %st = linalg.reduce {red_kind2} ins(%pt) outs(%a)
  scf.yield %st
}
```

Better: R1 is computed once, no recompute

Not the best: x is read twice ⇒ costly when the reduction dim is huge

Can we do better? Online Softmax

$$m = \max_j x_j$$

$$s = \sum_j e^{x_j - m}$$

$$\mathrm{softmax}(x)_j = \frac{e^{x_j - m}}{s}$$

$$e^{x_j - m_{\text{new}}} = e^{x_j - m_{\text{old}}} \cdot e^{m_{\text{old}} - m_{\text{new}}}$$

```
m = -inf;  s = 0
for each tile x_t of x:
  m_new = max(m, max(x_t))
  s = s * exp(m - m_new)     // correction
      + sum(exp(x_t - m_new))
  m = m_new
```

One pass over `x`: rescale the running sum whenever the max changes

When/Why does this work?

$$m = \max_j x_j$$

$$\text{softmax}(x)_j = \frac{e^{x_j - m}}{s}$$

$$s = \sum_j e^{x_j - m} = \sum_j \underbrace{e^{x_j}}_{\text{data}} \cdot \underbrace{e^{-m}}_{\text{R1}}$$

Stage $i-1$

$$s_{i-1} = \sum_{j<i} e^{x_j - m_{i-1}} \tag{1}$$

$$= e^{x_1} e^{-m_{i-1}} + e^{x_2} e^{-m_{i-1}} + \cdots + e^{x_{i-1}} e^{-m_{i-1}} \tag{2}$$

Stage i

$$m_i = \max(m_{i-1}, x_i) \tag{3}$$

$$s_{i-1} \cdot \frac{1}{e^{-m_{i-1}}} = e^{x_1} + e^{x_2} + \cdots + e^{x_{i-1}} \tag{4}$$

$$s_{i-1} \cdot \boxed{\frac{e^{-m_i}}{e^{-m_{i-1}}}} = e^{x_1 - m_i} + e^{x_2 - m_i} + \cdots + e^{x_{i-1} - m_i} \tag{5}$$

$$s_i = s_{i-1} \cdot e^{m_{i-1} - m_i} + e^{x_i - m_i} \tag{6}$$

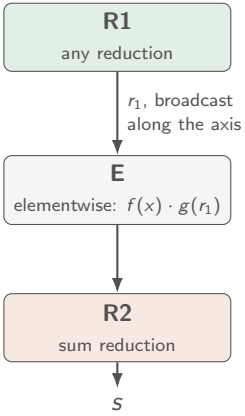
correction factor

R2 is a *sum*, and each summed term is $f(x) \cdot g(r_1)$ (multiplicatively separable)
Multiplication distributes over addition $\Rightarrow$ one rescale corrects every term

Legality Conditions

For a reduction chain R1 $\rightarrow$ E $\rightarrow$ R2:

1. **R1** can be *any* reduction
2. **E** (elementwise) uses R1's result *broadcast* along R1's reduction axis
3. E's use of R1 is *multiplicatively separable*: $E(x, r_1) = f(x) \cdot g(r_1)$
4. **R2** is a *sum* reduction that reduces over the same axis as R1



Multiplicative Separability

A lightweight *dataflow analysis* over E's body proves $E(x, r_1) = f(x) \cdot g(r_1)$

Facts, propagated from inputs to yield

- {IND}: independent of R1
- {ADD}: R1 enters *additively*
- {MUL}: R1 enters *multiplicatively*

Transfer rules

`add, sub`	keep {ADD}
`mul, div`	keep {MUL}
`exp`	{ADD} → {MUL}
`neg` / `abs`	keep all / keep {MUL}
other ops	no facts

Example: softmax, $E = \exp(x - m)$

```
// %x is {IND}, %m is {ADD, MUL}
%d = arith.subf %x, %m    // {ADD}
%e = math.exp %d          // {MUL}
linalg.yield %e           // {MUL}
```

Yield is {MUL} and depends on `%m` ⇒ separable ✓

Accept E only if its yield is {MUL} and depends on R1

Deriving Online Correction

1. Separability gives $E(x, r_1) = f(x) \cdot g(r_1)$. To isolate r_1's effect, build $E'(r_1) = g(r_1)$ by making $f(x) = 1$
2. Compute the *forward slice* of r_1 in E's body
3. Set every slice operand that does not depend on r_1 to the op's *neutral element* (0 for `add`/`sub`, 1 for `mul`/`div`)
4. Evaluate E' at r_1^{old} and r_1^{new}: correction $= E'(r_1^{\text{new}})/E'(r_1^{\text{old}})$

Example: $E(x, r_1) = \exp(x - r_1)$

```
// E

%d = arith.subf %x, %r1
%e = math.exp %d
linalg.yield %e
```

```
// E': %x := 0.0
%c0 = arith.constant 0.0
%d = arith.subf %c0, %r1
%e = math.exp %d
linalg.yield %e
```

$$E'(r_1) = \exp(0.0 - r_1)$$

$$\text{correction} = \frac{\exp(0.0 - r_1^{\text{new}})}{\exp(0.0 - r_1^{\text{old}})}$$

Proposed op: `structured.fuse_dependant_reduction_ops`

Three operand handles, one result:

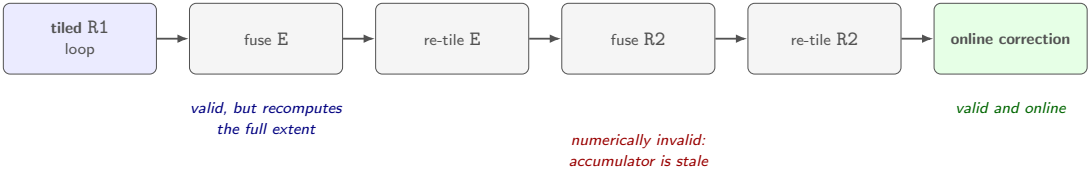
1. `R1` — already tiled producer reduction (`scf.for` from `tile_using_for` with `FullReduction`, accumulator carried as `iter_arg`; may carry several results).
2. `E` — all-parallel elementwise op consuming an `R1` result as a broadcast input.
3. `R2` — still-untiled consumer reduction over the same axis, reducing `E`'s result.

```
%loop = ..... -> !transform.any_op   // R1: the already tiled reduction loop
%e    = ..... -> !transform.any_op   // E : the elementwise term
%r2   = ..... -> !transform.any_op   // R2: the consumer reduction

%fused = transform.structured.fuse_dependant_reduction_ops %e, %r2 into %loop
    : (!transform.any_op, !transform.any_op, !transform.any_op)
    -> !transform.any_op
```

Returns the loop, now also carrying `E` and `R2`, fused and online ⇒ apply repeatedly for chains of dependent reductions.

Main stages of the transformation



- One op at a time, E first: *fuse* (`scf::tileAndFuseConsumer`), then immediately *re-tile* to R1's tile.
- E first, so R2 is fused against the slice E yields.
- Implemented as an `OpRewritePattern` on R2; a control function confines fusion to the handles named by the transform op.

Example: standard attention $\rightarrow$ fused attention

The attention chain

Score matrix `x = Q @ K^T`, values `V`; all reductions over the key axis `j`. Normalization is deferred (flash-attention-v2):

$$
\begin{aligned}
R1\ (\max): &\quad m[i] = \max_j x[i,j] \\
E\ (\text{term}): &\quad p[i,j] = \exp(x[i,j] - m[i]) \\
R2a\ (\text{sum}): &\quad s[i] = \textstyle\sum_j p[i,j] \\
R2b\ (P@V): &\quad o'[i,d] = \textstyle\sum_j p[i,j]\, V[j,d] \\
\text{div}: &\quad o[i,d] = o'[i,d]/s[i]
\end{aligned}
$$

- One term `E` feeds two consumer reductions over the same axis $\Rightarrow$ the op is applied twice.
- Shapes in the listings: `x` is `[64, 512]`, `V` is `[512, 128]`, reduction tile is 32.

Step 1: input IR (R1 already tiled + annotated)

```
// R1: the row max, tiled along the reduction axis and annotated.
%m = scf.for %iv = 0 to 512 step 32 iter_args(%acc_m = %ninf_64) {
  %xt   = tensor.extract_slice %x[0, %iv] [64, 32]     // score tile
  %mold = tensor.extract_slice %acc_m[0] [64]
  %mt   = linalg.generic ins(%xt) outs(%mold)          // max(x_tile, acc_m)
  %im   = tensor.insert_slice %mt into %acc_m[0] [64]
  scf.yield %im
} {__reduction_loop__}
// E: the term, at the full [64, 512] extent, read by BOTH reductions below.
%p = linalg.generic ins(%x, %m) outs(%empty_64x512)    // exp(x - m)
// R2a: the row sum of E.
%s = linalg.generic ins(%p) outs(%zero_64)             // sum_j p
// R2b: the P @ V contraction over E, reducing the same axis.
%o = linalg.generic ins(%p, %V) outs(%zero_64x128)     // sum_j p * V
// The deferred normalization, downstream of both reductions.
%out = linalg.generic ins(%o, %s) outs(%empty_64x128)  // o / s
```

Step 1: the transform schedule (one chain per application)

```
%gen = transform.structured.match ops{["linalg.generic"]} in %arg0 .....
// #1 = max (R1), #2 = E, #3 = row sum (R2a), #4 = contraction (R2b),
// #5 = the normalizing divide.
%r1, %e, %r2a, %r2b, %div = transform.split_handle %gen .....
%loop = transform.structured.match ops{["scf.for"]} in %arg0 .....

// Chain 1: fuse E and the row sum into the tiled max loop.
%fa = transform.structured.fuse_dependant_reduction_ops %e, %r2a into %loop .....
// The fused loop is the producer reduction of the second chain.
transform.annotate %fa "__reduction_loop__" : !transform.any_op

// The first fusion consumed the handle to E; the original E is still the
// contraction's operand, so re-derive it from there.
%e2 = transform.get_producer_of_operand %r2b[0] .....
// Chain 2: fuse E and the P @ V contraction into the same loop.
%fb = transform.structured.fuse_dependant_reduction_ops %e2, %r2b into %fa .....
```

Step 2: E fused — still at full extent

```
// A second iter_arg (%acc_p) now carries E's full-extent result.
%r:2 = scf.for %iv = 0 to 512 step 32 iter_args(%acc_m = %ninf_64, %acc_p = %empty_64x512) {
  %xt   = tensor.extract_slice %x[0, %iv] [64, 32]
  %mold = tensor.extract_slice %acc_m[0] [64]
  %mt   = linalg.generic ins(%xt) outs(%mold)          // max(x_tile, acc_m)
  // The fused clone of E: reads all 512 columns, writes the whole extent.
  %xf   = tensor.extract_slice %x[0, 0] [64, 512]
  %pdst = tensor.extract_slice %acc_p[0, 0] [64, 512]
  %pt   = linalg.generic ins(%xf, %mt) outs(%pdst)     // exp(x - m_new), FULL row
  %im   = tensor.insert_slice %mt into %acc_m[0] [64]
  %ip   = tensor.insert_slice %pt into %acc_p[0, 0] [64, 512]
  scf.yield %im, %ip
}
// ... still outside the loop: the row sum (now reading %r#1), the original E,
// the P @ V contraction and the normalizing divide.
```

E also feeds the contraction ⇒ a *clone* is fused; the original stays outside for the second application.

Step 3: E re-tiled to the reduction tile

```
%r:2 = scf.for %iv = 0 to 512 step 32 iter_args(%acc_m = %ninf_64, %acc_p = %empty_64x512) {
  %xt   = tensor.extract_slice %x[0, %iv] [64, 32]
  %mold = tensor.extract_slice %acc_m[0] [64]
  %mt   = linalg.generic ins(%xt) outs(%mold)          // max(x_tile, acc_m)
  // E, cut to the tile: reads the [64, 32] tile of x, writes a [64, 32] tile.
  %pdst = tensor.extract_slice %acc_p[0, %iv] [64, 32]
  %pt   = linalg.generic ins(%xt, %mt) outs(%pdst)     // exp(x_tile - m_new)
  %im   = tensor.insert_slice %mt into %acc_m[0] [64]
  %ip   = tensor.insert_slice %pt into %acc_p[0, %iv] [64, 32]
  scf.yield %im, %ip
}
```

- E is all-parallel, so its *destination* shrinks too: result type becomes [64, 32] and the yielding insert_slice is re-sliced to match.
- Numerically invalid from here: earlier tiles of the carried p were computed against older maxes.

Step 4: the row sum fused (and re-tiled)

```
%r:3 = scf.for %iv = 0 to 512 step 32
         iter_args(%acc_m = %ninf_64, %acc_p = %empty_64x512, %acc_s = %zero_64) {
  %xt   = tensor.extract_slice %x[0, %iv] [64, 32]
  %mold = tensor.extract_slice %acc_m[0] [64]
  %mt   = linalg.generic ins(%xt) outs(%mold)          // max(x_tile, acc_m)
  %pdst = tensor.extract_slice %acc_p[0, %iv] [64, 32]
  %pt   = linalg.generic ins(%xt, %mt) outs(%pdst)     // exp(x_tile - m_new)
  // The fused row sum, reducing this tile into the (stale) running sum.
  %sold = tensor.extract_slice %acc_s[0] [64]
  %st   = linalg.generic ins(%pt) outs(%sold)          // acc_s + sum_j p_tile
  ... yields: %mt, %pt, %st
}
```

Re-tiling only rewrites input slices carrying the reduction axis. For the row sum this is a no-op (its only input is `E`'s tile); for the `P @ V` contraction it is what cuts `V` from `[512, 128]` to `[32, 128]`.

Step 5: online correction — chain 1 complete

```
%r:3 = scf.for %iv = 0 to 512 step 32
         iter_args(%acc_m = %ninf_64, %acc_p = %empty_64x512, %acc_s = %zero_64) {
  %mt   = linalg.generic ins(%xt) outs(%mold)          // max(x_tile, acc_m)
  %pt   = linalg.generic ins(%xt, %mt) outs(%pdst)     // exp(x_tile - m_new)
  %sold = tensor.extract_slice %acc_s[0] [64]
  // The correction term: E isolated on the max (x := 0), evaluated at the new
  // max and at the previous running max.
  %tnew = linalg.generic ins(%mt) outs(%empty_64)      // exp(0 - m_new)
  %told = linalg.generic ins(%mold) outs(%empty_64)    // exp(0 - m_old)
  %f    = linalg.elementwise <div> ins(%tnew, %told)   // exp(m_old - m_new)
  // The rescaled running sum becomes the fused sum's DPS init.
  %sc   = linalg.elementwise <mul> ins(%sold, %f)      // acc_s * factor
  %st   = linalg.generic ins(%pt) outs(%sc)            // sc + sum_j p_tile
  ... yields: %mt, %pt, %st
}
```

The loop is again a tiled reduction loop ⇒ it can be the producer of the next chain.

Step 6: second application — fused loop (1/2)

```
%r:5 = scf.for %iv = 0 to 512 step 32
        iter_args(%acc_m = %ninf_64, %acc_p1 = %empty_64x512, %acc_s = %zero_64,
                  %acc_p2 = %empty_64x512, %acc_o = %zero_64x128) {
  %xt   = tensor.extract_slice %x[0, %iv] [64, 32]
  %mold = tensor.extract_slice %acc_m[0] [64]
  %mt   = linalg.generic ins(%xt) outs(%mold)           // max(x_tile, acc_m)
  // One E per fused chain: the clone for the row sum, the original for P @ V.
  %p1dst = tensor.extract_slice %acc_p1[0, %iv] [64, 32]
  %p1    = linalg.generic ins(%xt, %mt) outs(%p1dst)    // exp(x_tile - m_new)
  %p2dst = tensor.extract_slice %acc_p2[0, %iv] [64, 32]
  %p2    = linalg.generic ins(%xt, %mt) outs(%p2dst)    // exp(x_tile - m_new)

  // Chain 1: correction over the row sum's [64] accumulator shape.
  %sold = tensor.extract_slice %acc_s[0] [64]
  %tnew = linalg.generic ins(%mt) outs(%empty_64)       // exp(0 - m_new)
  %told = linalg.generic ins(%mold) outs(%empty_64)     // exp(0 - m_old)
  %f1   = linalg.elementwise <div> ins(%tnew, %told)    // exp(m_old - m_new)
  %sc   = linalg.elementwise <mul> ins(%sold, %f1)      // acc_s * factor
  %st   = linalg.generic ins(%p1) outs(%sc)             // sc + sum_j p1
```

Step 6: second application — fused loop (2/2)

```
  // Chain 2: the same term, correction over the [64, 128] accumulator shape.
  %vt    = tensor.extract_slice %V[%iv, 0] [32, 128]     // value tile
  %oold  = tensor.extract_slice %acc_o[0, 0] [64, 128]
  %tnew2 = linalg.generic ins(%mt) outs(%empty_64x128)   // exp(0 - m_new), broadcast
  %told2 = linalg.generic ins(%mold) outs(%empty_64x128) // exp(0 - m_old), broadcast
  %f2    = linalg.elementwise <div> ins(%tnew2, %told2)  // exp(m_old - m_new)
  %oc    = linalg.elementwise <mul> ins(%oold, %f2)      // acc_o * factor
  %ot    = linalg.generic ins(%p2, %vt) outs(%oc)        // oc + sum_j p2 * V_tile

  %im  = tensor.insert_slice %mt into %acc_m[0] [64]
  %ip1 = tensor.insert_slice %p1 into %acc_p1[0, %iv] [64, 32]
  %is  = tensor.insert_slice %st into %acc_s[0] [64]
  %ip2 = tensor.insert_slice %p2 into %acc_p2[0, %iv] [64, 32]
  %io  = tensor.insert_slice %ot into %acc_o[0, 0] [64, 128]
  scf.yield %im, %ip1, %is, %ip2, %io
}
// Only the normalizing divide is left outside.
%out = linalg.generic ins(%r#4, %r#2) outs(%empty_64x128)  // o / s
```

One online loop, five accumulators: running max, one `E` tile buffer per chain, running sum, running `P @ V`.

Evaluation

Experimental setup: attention

Standard attention generated from Linalg-level ops, lowered to the XeGPU backend and run end-to-end on GPU hardware.

- **Hardware**: Intel Battlemage (BMG) GPU.
- **Sizes**: `Z = 2` (batch), `H = 8` heads, head dim = `64`, `f16`; `n_ctx` $\in \{1024, 4096, 8192, 16384\}$.
- **Baseline (unfused)**: same kernel tiled/fused with existing upstream transforms only — no fusion on the reduction dim, score matrix recomputed for each reduction.
- **This RFC**: schedule from this proposal, fusing the max, sum and `attn * V` reductions into a single loop.

Results: attention (BMG)

n_ctx	unfused	reduction fusion (this RFC)
1024	1.00x	**2.49x**
4096	1.00x	**2.69x**
8192	1.00x	**2.59x**
16384	1.00x	**2.19x**

Fusing the dependent reductions into a single online loop gives a 2.2x–2.7x speedup across the sweep.

Results: softmax and L1/L2 normalization (BMG)

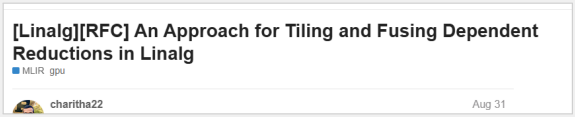
One application of the op each; the epilogue recomputes its term from the *final* max, so three passes over the input become two. The three differ only in `E`, and so in the correction — no code specific to `exp`:

kernel	`E`	correction derived from `E`
softmax	`exp(x - m)`	`exp(m_old - m_new)`
L1 normalize	`abs(x / m)`	`m_old / m_new`
L2 normalize	`(x / m)^2`	`(m_old / m_new)^2`

reduction dim	baseline	softmax	L1 norm.	L2 norm.
4096	1.00x	**1.23x**	**1.22x**	**1.22x**
8192	1.00x	**1.23x**	**1.22x**	**1.23x**
16384	1.00x	**1.21x**	**1.20x**	**1.21x**

`f32`, parallel dim 4096; wg 64 / sg 8 rows, sg size 16; `fastmath<fast>`, best tile per variant (128 / 64). Normalization uses the overflow-safe max-scaled form (BLAS `nrm2`) — hence the max. All three land within 0.01x at every size: same loop, only `E` differs.

Current Status



LLVM Lighthouse contributions

[linalg][transforms] Add legality checks for fusing dependent reductions. #295

Merged tkarna merged 10 commits into llvm:main from charithaintc:dependant_reduction_legality 3 weeks ago

[linalg][transfrom] A transform extension for tiling and fusing dependent reductions. #297

Merged tkarna merged 7 commits into llvm:main from charithaintc:linalg_reduction_fusion 3 weeks ago

Thank you!

Questions?